

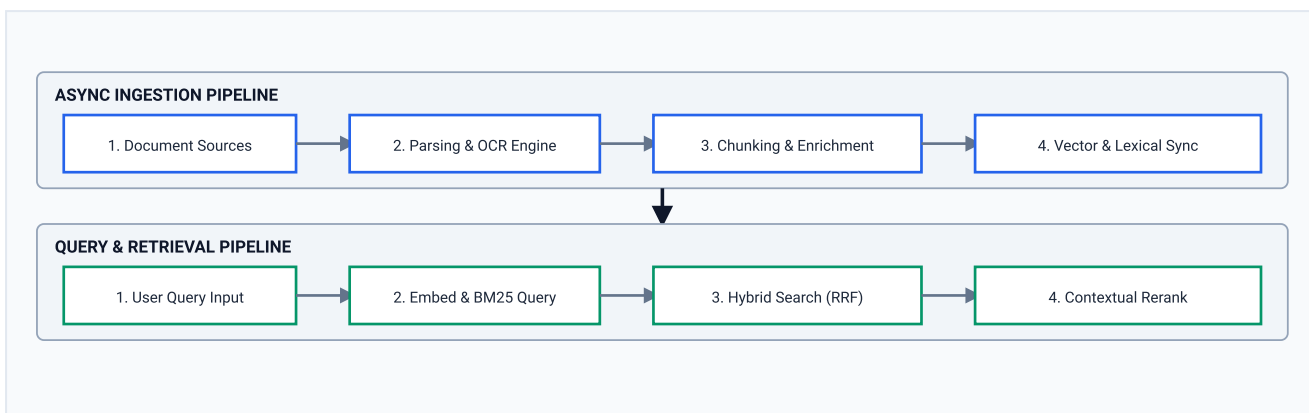
Enterprise Search Engine & Ingestion Pipeline

TECHNICAL ARCHITECTURE & HIGH-PERFORMANCE IMPLEMENTATION GUIDE

Executive Summary: Modern enterprise document ingestion engines pair high-throughput async processing pipelines with a unified hybrid search architecture (Dense Vector Semantic Search + Sparse BM25 Lexical Keyword Search) to deliver sub-second search across unstructured document stores.

1. System Architecture Overview

The system is split into two asynchronous workflows: the continuous **Ingestion Pipeline** for ingestion, OCR, parsing, and dual-indexing, and the synchronous **Query & Retrieval Pipeline** for query expansion, candidate retrieval, and cross-encoder reranking.



2. Core Technical Components

1. Document Connectors

Hooks into AWS S3, Google Drive, SharePoint, and relational databases. Uses message brokers (Kafka/RabbitMQ) for real-time change detection.

2. OCR & Structural Parsing

Extracts text and tabular content using Layout-Aware Vision Models (Textract, Unstructured, LlamaParse) without losing table integrity.

3. Semantic Chunking Strategy

Splits text into 500-token passages with a 50-token overlap. Attaches metadata (ACLs, doc_id, timestamps) for downstream filtering.

4. Dense & Sparse Dual Indexing

Stores 384-dim dense vector embeddings in Qdrant/Pinecone while simultaneously indexing exact tokens into a BM25 inverted index.

3. Recommended Technical Stack

Selection matrix comparing open-source frameworks against enterprise managed cloud platforms:

Domain Layer	Open-Source / Self-Hosted	Fully Managed Cloud
Ingestion & Task Queue	Apache Airflow, Temporal, Celery	AWS Step Functions, Azure Durable Functions
OCR & Text Parsing	Tesseract OCR, Unstructured, LlamaParse	AWS Textract, Azure Document Intelligence
Vector Store Engine	Qdrant, Milvus, pgvector	Pinecone, AWS OpenSearch Serverless
Lexical Inverted Index	Elasticsearch, OpenSearch	Azure AI Search, Elastic Cloud
Embeddings & Reranker	FastEmbed, BGE-Large, SentenceTransformers	OpenAI Embeddings, Cohere Rerank API

4. Pipeline Implementation Engine

Below is the complete, runnable Python implementation featuring asynchronous ingestion, chunking, dense vector indexing via Qdrant, lexical index construction via BM25, and Reciprocal Rank Fusion (RRF) search capabilities.

```
import hashlib from typing import List, Dict, Any from langchain_text_splitters import RecursiveCharacterTextSplitter from qdrant_client import QdrantClient from qdrant_client.models import Distance, VectorParams, PointStruct from fastembed import TextEmbedding from rank_bm25 import BM25Okapi class IngestionEngine: """Handles document hashing, parsing, chunking, and vector indexing.""" def __init__(self, collection_name: str = "documents"): self.collection_name = collection_name self.embedding_model = TextEmbedding(model_name="BAAI/bge-small-en-v1.5") self.qdrant = QdrantClient(":memory:") self.text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50, length_function=len) self._init_collection() def _init_collection(self): self.qdrant.recreate_collection(collection_name=self.collection_name, vector_params=VectorParams(size=384, distance=Distance.COSINE)) def process_and_index(self, doc_id: str, raw_text: str, metadata: Dict[str, Any]): chunks = self.text_splitter.split_text(raw_text) embeddings = list(self.embedding_model.embed(chunks)) points = [] for idx, (chunk, embedding) in enumerate(zip(chunks, embeddings)): point_id = hashlib.md5(f"{doc_id}_{idx}".encode()).hexdigest() chunk_metadata = {**metadata, "doc_id": doc_id, "chunk_index": idx, "text": chunk} points.append(PointStruct(id=point_id, vector=embedding.tolist(), payload=chunk_metadata)) self.qdrant.upsert(collection_name=self.collection_name, points=points) return chunks
```

4. Pipeline Implementation Engine (Continued)

```
class HybridSearchEngine: """Performs Hybrid Search using Dense Vector Search + Lexical BM25 + RRF."""
    def __init__(self, ingestion_engine: IngestionEngine):
        self.ingestion = ingestion_engine
        self.corpus_chunks: List[Dict[str, Any]] = []
        self.bm25 = None

    def build_bm25_index(self, all_chunks_payload: List[Dict[str, Any]]):
        self.corpus_chunks = all_chunks_payload
        tokenized_corpus = [doc["text"].lower().split() for doc in self.corpus_chunks]
        self.bm25 = BM25Okapi(tokenized_corpus)

    def dense_search(self, query: str, top_k: int = 5) -> List[Dict[str, Any]]:
        query_vector = list(self.ingestion.embedding_model.embed([query]))[0].tolist()
        search_result = self.ingestion.qdrant.search(collection_name=self.ingestion.collection_name, query_vector=query_vector, limit=top_k)
        return [hit.payload for hit in search_result]

    def lexical_search(self, query: str, top_k: int = 5) -> List[Dict[str, Any]]:
        tokenized_query = query.lower().split()
        return self.bm25.get_top_n(tokenized_query, self.corpus_chunks, n=top_k)

    def hybrid_search(self, query: str, top_k: int = 3, rrf_k: int = 60) -> List[Dict[str, Any]]:
        dense_hits = self.dense_search(query, top_k=top_k * 2)
        lexical_hits = self.lexical_search(query, top_k=top_k * 2)
        rrf_scores: Dict[str, float] = {}
        doc_store: Dict[str, Dict[str, Any]] = {}
        for rank, hit in enumerate(dense_hits):
            key = f"{hit['doc_id']}_{hit['chunk_index']}"
            doc_store[key] = hit
            rrf_scores[key] = rrf_scores.get(key, 0.0) + (1.0 / (rrf_k + rank + 1))
        for rank, hit in enumerate(lexical_hits):
            key = f"{hit['doc_id']}_{hit['chunk_index']}"
            doc_store[key] = hit
            rrf_scores[key] = rrf_scores.get(key, 0.0) + (1.0 / (rrf_k + rank + 1))
        sorted_keys = sorted(rrf_scores.keys(), key=lambda k: rrf_scores[k], reverse=True)
        return [doc_store[k] for k in sorted_keys[:top_k]]

    # Execution Example
    if __name__ == "__main__":
        engine = IngestionEngine()
        engine.process_and_index("doc_1", "Role-Based Access Control and vector security.", {"title": "Security"})
        print("Ingestion & Hybrid Pipeline Initialized Successfully.")
```

5. Key Enterprise Considerations

- **Role-Based Access Control (RBAC):** Enforce payload pre-filtering during dense search queries to prevent security leaks between tenants.
- **Content Deduplication:** Compute SHA-256 document signatures during the ingestion phase to skip re-chunking duplicate files.
- **Change Data Capture (CDC):** Ensure event listeners immediately remove orphaned vector and lexical index entries when documents are modified or deleted.

